One Kernel, Two Readers: Graph-Native Proximity Search as Direct Evaluation of the Kernel that Embeddings Factorize

July 2026 — draft

Status: reference material — there is no behavioral contract here. GitHub's markdown math rendering is unreliable for this document — read the typeset **PDF version** instead. Companion note: illuminate-complexity.md compares the same traversal against B-tree self-joins; this note compares it against embedding + approximate-nearest-neighbor (ANN) retrieval. A paper-style treatment of the same mathematics framed for the music / information-recommendation literature is music-recommendation-walk-kernels.md. Code anchors: core/graphcache/traversal.go (`Neighbor`, `PersonalizedPageRank`, `LocalCommunity`), core/graphcache/edge.go (decaying weights).

Abstract. Lantern answers "what is near this vertex?" by walking a live, time-decaying weighted graph. Embedding-based retrieval answers the same question by mapping items into $\mathbb{R}^d$ and running (approximate) nearest-neighbor search. We make precise the sense in which these are two estimators of the *same* mathematical object — a random-walk proximity kernel $M = \sum_k c_k P^k$ over the item set — and locate exactly where the correspondence is a theorem and where it breaks. A line of results (Levy–Goldberg [12], NetMF [22], HOPE [21]) shows that the most widely used embedding families are implicit or explicit low-rank factorizations of such kernels, so "embedding + vector search" reads a rank-d approximation $\Phi\Phi^\top \approx M$; Lantern's Personalized-PageRank operator instead evaluates a row of M directly by local forward-push, with total work $O(1/(\alpha\varepsilon))$ *independent of graph size* [5]. Conversely, the dominant ANN indexes are themselves greedy graph traversals [26], so each side implements the other. From this single structural difference — *factorize once, read the factors* versus *never factorize, read the kernel at query time* — we derive the three practical consequences observed in Lantern: (i) a cost model with $O(1)$ writes and size-independent local reads, (ii) cold-start exactness at any sample size, below the sample-complexity threshold where low-rank recovery is information-theoretically impossible [29, 30], and (iii) native handling of non-stationarity, where the kernel is evaluated at read time on TTL-decayed weights while a factorization serves a frozen snapshot. We also state the honest converses: rank obstructions cut both ways [32, 33], and the low-rank reader's ability to *complete* unobserved entries is a generalization capability that direct evaluation deliberately does not have.

---

1. Introduction

Two families of systems answer proximity queries over a set of items V:

- **Embedding + ANN.** Choose an encoder $\varphi : V \to \mathbb{R}^d$, store $\{\varphi(v)\}_{v \in V}$ in an ANN index, and answer a query q with $\arg\text{top-}k_v \langle \varphi(q), \varphi(v) \rangle$ (or cosine / negative distance).
- **Graph-native proximity.** Maintain an explicit weighted directed graph on V and answer with a bounded traversal from the query vertex: beam-pruned BFS, Personalized PageRank (PPR), or a local community around the seed. This is Lantern's model: the graph is the mathematical object $G = (V, E, w)$ — weights are the only edge decoration — and vertices/edges decay under TTLs.

These look like different retrieval philosophies. The observation this note develops is that they are **two readers of one object**: a *walk kernel* $M = \sum_{k \geq 0} c_k P^k$ built from the transition operator P of a graph.

1. Modern embeddings are (implicit or explicit) **low-rank factorizations of walk kernels**. This is not an analogy; it is a sequence of theorems ([12], [22], [21], §3).
2. Lantern's PPR operator is a **direct, local, query-time evaluation** of one row of the resolvent kernel $\alpha(I - (1-\alpha)P)^{-1}$, with an $O(1/(\alpha\varepsilon))$ work bound independent of $|V|, |E|$ ([5], §4).
3. The two readers agree precisely on the regime where M is near-low-rank and near-symmetric; the divergences — rank obstructions, asymmetry, completion of unobserved entries — are characterizable, and each is a feature for one side and a limitation for the other (§5).
4. Because the factorization step is what costs a global computation, demands a minimum sample size, and freezes time, deleting that step yields the three practical properties Lantern exhibits: cheap dynamics, cold-start exactness, and freshness under drift (§6).

Throughout, we keep the mathematics at the level of precise statements with short proofs where they are genuinely short, and citations where they are not.

2. The object of study: a time-indexed proximity kernel

2.1 Lantern's graph

Let V be a finite set of vertices. Lantern's edge weights are sums of additive, individually expiring contributions (edge.go):

Definition 2.1 (decaying weight). An edge (u, v) carries a finite multiset of contributions $\{(c_i, e_i)\}_i$ with values $c_i \in \mathbb{R}$ and expirations $e_i \in \mathbb{R}$. Its weight at time t is

$$w_t(u, v) \;=\; \sum_i c_i \, \mathbf{1}\{t < e_i\}.$$

Thus $t \mapsto w_t(u, v)$ is a right-continuous step function decaying to 0 after the last expiration: each entry of the weight matrix is a **sliding-window sum of evidence** (a time-decayed stream aggregate in the sense of [41]). A write appends one contribution — an $O(1)$ local operation — and reads evaluate the sum at the single instant `now` chosen once per traversal.

The graph at time t is $G_t = (V, E_t, w_t)$ with $E_t = \{(u, v) : w_t(u, v) \neq 0\}$, directed and weighted; there are no edge labels or properties.

2.2 Reweighting and the transition operator

Before any traversal, Lantern optionally re-scores raw weights (`EdgeWeighting` in traversal.go): with $\mathrm{df}(v)$ the in-degree of v counted over distinct tails,

$$\psi_{\mathrm{raw}}(w) = w, \qquad \psi_{\mathrm{tfidf}}(w \mid v) = \frac{w}{\log_2(1 + \mathrm{df}(v))}, \qquad \psi_{\mathrm{bm25}}(w \mid v) = \mathrm{IDF}(\mathrm{df}(v)) \cdot \frac{w\,(k_1 + 1)}{w + k_1(1 - b + b\,\frac{|u|}{\overline{|u|}})},$$

the last being Okapi BM25 [17] with term frequency = the live edge weight, document length $|u|$ = the tail's out-degree, and corpus statistics over tails. Write A_t^{ψ} for the reweighted adjacency matrix, $d^{\psi}(u) = \sum_v A_t^{\psi}(u, v)$ for the weighted out-degree, and define the (sub)stochastic transition matrix

$$P(u, v) \;=\; \begin{cases} A_t^{\psi}(u, v)/d^{\psi}(u) & d^{\psi}(u) > 0, \\ 0 & \text{otherwise (sink: the row is zero and walk mass dies).} \end{cases}$$

P is row-substochastic; rows of sinks are zero, matching the implementation, where a sink absorbs the restart share of its residual and leaks the remainder.

2.3 Walk kernels

Definition 2.2 (walk kernel). Given nonnegative coefficients (c_k) with $\sum_k c_k < \infty$, the associated kernel is

$$M \;=\; \sum_{k\geq 0} c_k\, P^k.$$

$M(u, v)$ aggregates the probability of reaching v from u over walks of every length, discounted by c_k. The classical members:

Kernel	Coefficients	Closed form
Katz index [1]	$c_k = \beta^k$ on A rather than P	$(I - \beta A)^{-1} - I$, for $\beta < 1/\rho(A)$
Personalized PageRank [2, 3, 4]	$c_k = \alpha(1-\alpha)^k$	$\alpha\,(I - (1-\alpha)P)^{-1}$
Heat / diffusion kernel [8]	$c_k = e^{-\beta}\beta^k/k!$	$e^{-\beta(I-P)}$
DeepWalk window kernel [22]	$c_k = \frac{1}{T}\mathbf{1}\{1 \leq k \leq T\}$	$\frac{1}{T}\sum_{r=1}^{T} P^r$

Proposition 2.3 (PPR is well defined). For $\alpha \in (0, 1)$ the series $\alpha \sum_{k\geq 0}(1-\alpha)^k P^k$ converges and equals $\alpha(I - (1-\alpha)P)^{-1}$.

Proof. P is substochastic, so $\|P\|_\infty \leq 1$ and $\|(1-\alpha)P\|_\infty \leq 1-\alpha < 1$; the Neumann series of $I-(1-\alpha)P$ converges in operator norm. □

We write $\pi_s = \alpha\, e_s^\top (I-(1-\alpha)P)^{-1}$ for the PPR row seeded at s; equivalently π_s is the unique fixed point of $\pi_s = \alpha\, e_s^\top + (1-\alpha)\,\pi_s P$. Probabilistically, $\pi_s(v)$ is the stationary mass at v of a walker who restarts at s with probability α at each step; $\|\pi_s\|_1 \leq 1$ with equality iff no mass dies in sinks.

Everything downstream concerns two ways of *reading a row* of some M: given s, return $\arg\text{top-}k_v M(s, v)$.

3. Reader I: embedding + ANN as low-rank factorization

The embedding pipeline is: (i) learn or compute $\Phi \in \mathbb{R}^{|V|\times d}$, (ii) index the rows, (iii) at query time return the top-k inner products. Step (iii) reads a row of $\Phi\Phi^\top$. The content of this section is that for the standard training objectives, $\Phi\Phi^\top$ is a low-rank approximation of a walk kernel — so the pipeline as a whole reads a row of $\widehat{M} \approx M$.

Theorem 3.1 (Levy–Goldberg [12]). Skip-gram with negative sampling (word2vec [11]) with k negative samples, trained to its global optimum with unconstrained dimension, produces word/context vectors whose inner products satisfy

$$\langle \varphi(u), \varphi_c(v)\rangle \;=\; \mathrm{PMI}(u, v) - \log k, \qquad \mathrm{PMI}(u, v) = \log \frac{\Pr(u, v)}{\Pr(u)\Pr(v)}.$$

That is, SGNS implicitly factorizes the shifted pointwise-mutual-information matrix [15]. With $d < |V|$, it computes a rank-d approximation of that matrix. GloVe [13] makes the log-co-occurrence factorization explicit, and [14] gives a generative model under which the PMI identity arises.

Theorem 3.2 (NetMF, Qiu et al. [22]). DeepWalk [18] with window T and b negative samples, in the limit of infinite random-walk corpus, implicitly factorizes the elementwise logarithm of the positive part of

$$\frac{\mathrm{vol}(G)}{bT}\left(\sum_{r=1}^{T} P^r\right) D^{-1},$$

where $\mathrm{vol}(G) = \sum_{u,v} A(u,v)$ and D is the degree matrix. LINE [19] is the special case $T = 1$; node2vec [20] factorizes the analogous expression for its second-order walk. In words: **the flagship graph-embedding methods are rank-d factorizations of (a log-transform of) the DeepWalk window kernel of Definition 2.2.**

Explicit constructions. HOPE [21] skips the sampling and directly factorizes kernels of the form $S = M_g^{-1} M_l$ — including Katz ($M_g = I - \beta A$, $M_l = \beta A$) and rooted PageRank (the PPR resolvent, up to reparameterization of α) — into *asymmetric* source/target factors $S \approx U^s (U^t)^\top$ via a partial generalized SVD. VERSE [23] trains embeddings whose inner-product softmax matches PPR rows; InstantEmbedding [24] computes a single vertex's embedding from one local PPR push plus hashing — literally compressing π_s into $\mathbb{R}^d$. Laplacian eigenmaps [25] is the spectral ancestor: eigenvectors of the random-walk Laplacian, i.e., the optimal factorization of a spectrally transformed P.

Remark 3.3 (optimality and error). For a symmetric kernel the best rank-d approximation in Frobenius/spectral norm is the truncated eigen- decomposition (Eckart–Young [50]), with error exactly the discarded spectral tail $\left(\sum_{i>d} \sigma_i^2\right)^{1/2}$. So the embedding reader's *systematic* error is governed by how fast the spectrum of M decays; the ANN index adds a *recall* error on top (it returns an approximate top-k).

The converse: ANN search is itself graph traversal. HNSW [26] and its navigable-small-world ancestor [27] answer nearest-neighbor queries by greedy beam search over a proximity graph constructed from the metric; theoretical guarantees for this family are analyzed in [28]. Two consequences:

Proposition 3.4 (exact simulation on kNN graphs). Let G_φ be the directed graph with an edge $u \to v$ of weight $\langle \varphi(u), \varphi(v) \rangle$ (shifted to be positive) whenever v is among the k exact nearest neighbors of u. Then Lantern's `Neighbor(seed, step=1, k, raw, largest)` on G_φ returns exactly the vector-search top-k of the seed.

Proof. Immediate from the per-hop top-k prune: the frontier of the seed is its stored kNN list, ranked by the stored weights. □

So graph traversal *subsumes* vector retrieval given the right graph, and practical vector retrieval *is* (approximate) graph traversal. The question "graph or vectors?" is therefore not about the search procedure at all; it is about **where the graph comes from** — asserted by writers as ground truth (Lantern), or reconstructed from a learned metric (ANN over embeddings) — and about whether a factorization sits between the data and the reader.

4. Reader II: Lantern evaluates the kernel directly

Lantern's three traversal operators, in kernel language:

- `Neighbor` (`step` hops, beam k): a greedy, per-hop-truncated reader of $\sum_{r \le \mathrm{step}} P^r$-type locality — algorithmically the same beam-pruned BFS that HNSW uses, run on the asserted graph. Cost analysis against relational engines: illuminate-complexity.md.
- `PersonalizedPageRank`: a certified reader of one row of $\alpha(I - (1-\alpha)P)^{-1}$, via Andersen–Chung–Lang (ACL) forward-push [5] (bookmark-coloring in Berkhin's terminology [6]).
- `LocalCommunity`: PageRank-Nibble [5] — the same push followed by a sweep cut minimizing conductance over prefixes of the push's support.

The push is where the mathematics is sharpest, so we state it fully. The algorithm maintains $p, r : V \to \mathbb{R}_{\ge 0}$ with $p = 0$, $r = e_s$ initially, and repeatedly picks any u with $r(u) \ge \varepsilon \cdot \max(\deg(u), 1)$ (where deg counts u's live, kept out-edges) and **pushes**:

$$p(u) \mathrel{+}= \alpha\, r(u), \qquad r(v) \mathrel{+}= (1-\alpha)\, r(u)\, P(u,v) \;\; \forall v, \qquad r(u) \leftarrow 0.$$

It terminates when no vertex exceeds its threshold and returns p as the estimate of π_s.

Lemma 4.1 (invariant). At every step of the algorithm,

$$\pi_s \;=\; p \;+\; \sum_{u \in V} r(u)\, \pi_u.$$

Proof. Initially $p = 0$, $r = e_s$, and the identity reads $\pi_s = \pi_s$. A push at u with residual $\rho = r(u)$ replaces the term $\rho\, \pi_u$ by $\alpha\rho\, e_u + (1-\alpha)\rho \sum_v P(u,v)\, \pi_v$ (the first summand entering p, the second entering the residuals). By the fixed-point identity $\pi_u = \alpha\, e_u + (1-\alpha) \sum_v P(u,v)\, \pi_v$, the right-hand side is unchanged. □

Since every $\pi_u \geq 0$, Lemma 4.1 gives at once $0 \leq p \leq \pi_s$ entrywise: the estimate never overshoots, and the error has the exact representation $\pi_s - p = \sum_u r(u)\pi_u$.

Theorem 4.2 (work bound, [5]). With threshold $\varepsilon \cdot \max(\deg(u), 1)$, the number of pushes is at most $1/(\alpha\varepsilon)$, and moreover $\sum_{\text{pushes}} \deg(u_i) \leq 1/(\alpha\varepsilon)$, so the total work is $O(1/(\alpha\varepsilon))$ — **independent of $|V|$ and $|E|$.**

Proof. A push at u with residual ρ changes $\|r\|_1$ by $-\rho + (1-\alpha)\rho \sum_v P(u,v) \leq -\alpha\rho$ (using substochasticity of the row). Since $\|r\|_1 = 1$ initially and $\|r\|_1 \geq 0$ always, and each push has $\rho \geq \varepsilon \max(\deg(u), 1) \geq \varepsilon$, the number of pushes is at most $1/(\alpha\varepsilon)$; and summing $\alpha\varepsilon \deg(u_i) \leq \alpha\rho_i$ over pushes gives $\sum_i \deg(u_i) \leq 1/(\alpha\varepsilon)$, which dominates the work since a push at u costs $O(\deg(u))$. □

This bound is the design basis of the implementation: the hard budget in `pprMaxPushes` (traversal.go) is a constant multiple of $1/(\alpha\varepsilon)$.

Theorem 4.3 (approximation, undirected case [5]). If the graph is undirected (w symmetric, ψ symmetric-preserving), then at termination

$$0 \;\leq\; \pi_s(v) - p(v) \;\leq\; \varepsilon\, d(v) \qquad \text{for every } v.$$

Proof. For reversible $P = D^{-1}A$ with symmetric A, the matrix $DP^k = (AD^{-1})^{k-1}A$ is symmetric for every k, hence so is $D\,\Pi$ where Π is the PPR kernel; entrywise this is the reciprocity identity $d(u)\, \pi_u(v) = d(v)\, \pi_v(u)$. Then by Lemma 4.1 and the termination condition $r(u) \leq \varepsilon\, d(u)$,

$$\pi_s(v) - p(v) \;=\; \sum_u r(u)\, \pi_u(v) \;\leq\; \varepsilon \sum_u d(u)\, \pi_u(v) \;=\; \varepsilon\, d(v) \sum_u \pi_v(u) \;\leq\; \varepsilon\, d(v). \;\square$$

Remark 4.4 (directed case). Lantern's graphs are directed, where the reciprocity identity fails. Lemma 4.1 and Theorem 4.2 hold verbatim (they use only linearity and substochasticity), so the estimate is still a certified underestimate with exact error representation and size-independent cost; the clean entrywise bound of Theorem 4.3 is specific to reversible chains and should be regarded as a heuristic in the directed setting. The sink rule in the implementation (degree floor 1; the restart share is banked, the continuation share leaks) is exactly the push for the substochastic P of §2.2, so no separate analysis is needed.

Sweep cut. `LocalCommunity` orders the push's support by $p(v)/\max(d^{\psi}(v), 1)$ and returns the prefix minimizing the (directed, weighted, out-volume) conductance

$$\phi(S) \;=\; \frac{\text{cut}(S)}{\min\left(\text{vol}(S),\, \text{vol}(\text{touched}) - \text{vol}(S)\right)},$$

the PageRank-Nibble procedure. For undirected graphs ACL prove, via the Lovász–Simonovits curve technique [7], that if the seed lies well inside a set of conductance ϕ, the sweep finds a set of conductance $O(\sqrt{\phi \log n})$ with the corresponding locality of work [5]. As with Theorem 4.3, Lantern inherits the *procedure* on directed weighted graphs and the *guarantee* on the symmetric restriction.

Kernel reweighting = the PMI lesson. Why does Lantern re-score weights with ψ_{tfidf} or ψ_{bm25} before walking? For the same reason word embeddings factorize PMI rather than raw co-occurrence counts, and search engines rank by IDF-weighted term frequency rather than raw frequency [15, 16, 17]: raw evidence is dominated by globally frequent items (hubs), and dividing out an occurrence prior converts "popular" into "informative". Formally both are entrywise transforms applied to the kernel's input matrix before the reader runs — $\log(\cdot) - \log k$ shift in SGNS (Theorem 3.1), $w/\log_2(1+\text{df})$ or BM25 in Lantern. The parallel is exact and, we think, clarifying: **the preprocessing that makes factorization meaningful and the preprocessing that makes traversal meaningful are the same operation.**

5. When the two readers agree — and when they cannot

5.1 The agreement regime

Fix a kernel M. If (i) M is exactly rank d and symmetric positive semidefinite, so $M = \Phi\Phi^\top$ exactly, and (ii) the ANN index has perfect recall, then the two readers return identical top-k sets for every seed, by construction. Perturbing (i) costs the spectral-tail error of Remark 3.3; perturbing (ii) costs index recall. So the meaningful question is not whether the readers *can* agree — on near-low-rank, near-symmetric kernels they provably do — but what happens outside that regime. Empirically the regime is substantial: this is *why* embedding methods work, and why proximity indices and latent-factor methods have traded blows on link prediction since [10].

5.2 Obstruction: rank and geometry

Rank. Seshadhri, Sharma, Stolman and Goel [32] prove that in the standard dot-product embedding model, low-dimensional factorizations cannot reproduce the combination of sparse (low average degree) and triangle-dense structure characteristic of real networks: any $d \ll \text{polylog}\, n$ loses either the degree distribution or the triangle density. Chanpuriya et al. [33] sharpen the picture: the obstruction is partly an artifact of that specific factorization model — with asymmetric factors and a logistic link, exact low-rank representations of sparse networks exist. The fair summary for our purposes: **whether M is representable in $\mathbb{R}^d$ depends delicately on the embedding model, and for the natural inner-product model there are explicit impossibility theorems.** A graph store carries M exactly and is indifferent to this question.

Asymmetry. $\langle \varphi(u), \varphi(v) \rangle$ is symmetric; Lantern's w_t and every kernel built on it are not. Asymmetric factorizations ($U^s(U^t)^\top$, HOPE [21]; word2vec's input/output vectors) recover directionality at the price of doubling parameters and losing the metric interpretation that ANN indexes exploit. Non-metric and asymmetric similarity is not exotic: it is the *empirically observed* structure of human similarity judgment (Tversky [34]); density/order/hyperbolic embeddings [35, 36] exist precisely because flat inner products fail on hierarchical data.

Intransitivity and composition. A walk kernel composes multiplicatively along paths: two strong hops yield a quantifiable two-hop score, and $M(u, v) = 0$ exactly when no path carries weight. Inner-product similarity does not compose: $\text{sim}(a, b)$ and $\text{sim}(b, c)$ constrain $\text{sim}(a, c)$ only weakly. The graph can assert "$a \sim b$, $b \sim c$, $a \nsim c$" at full sharpness; any rank-d factorization smooths it.

5.3 Obstruction and feature: completion versus fidelity

A rank-d factorization fitted to observed entries of M *fills in* the unobserved entries — this is exactly the mechanism of matrix completion [29, 31], and it is why embedding retrieval generalizes: it returns

similar-but-never-co-observed items. Direct evaluation returns 0 for pairs with no connecting path — no invented similarity, and no serendipity either.

The same theory prices this generalization. Exact (and stable) recovery of an incoherent rank-r kernel on n items requires on the order of $nr\,\mathrm{polylog}(n)$ observed entries, and $\Omega(nr \log n)$ is information-theoretically necessary [30]. Below that threshold the factorization is not merely inaccurate; the missing entries are non-identifiable. This is the precise form of the cold-start statement in §6.2: the regime where the low-rank reader is *statistically meaningless* is exactly the regime where the direct reader is *exact on everything asserted*.

6. Consequences: cost, cold start, non-stationarity

All three consequences follow from one structural fact: **the embedding pipeline contains a factorization stage and Lantern's pipeline does not.** The factorization is (a) a global computation, (b) a statistical estimator with a sample-complexity threshold, and (c) a snapshot taken at some time t_0. Deleting the stage deletes all three costs; the price paid is §5.3's generalization.

6.1 Cost model

	Lantern (direct reader)	Embedding + ANN (factorized reader)
Ingest one relation	append a contribution: amortized $O(1)$, local (edge.go)	encoder forward pass (O(model), typically accelerator-bound) + ANN insert ($O(d \cdot \mathrm{ef} \cdot \log n)$ empirically [26])
Query	push: total work $\leq 1/(\alpha\varepsilon)$, **independent of** $\lvert V\rvert, \lvert E\rvert$ (Thm 4.2); comparisons are scalar reads	encode the query (one forward pass) + ANN search (empirically $O(d\,\mathrm{ef} \log n)$ [26, 28]); every comparison costs $\Theta(d)$, $d \in [384, 3072]$ typical
Model refresh under drift	none exists	re-train / re-encode + re-index: global, $\Omega(n)$
Delete / forget one item's influence	wait for expiry, or delete the edge: $O(1)$	unlearning problem [39, 40]; streaming ANN deletion requires index consolidation [38]
Steady-state memory	$O(\lvert V\rvert + \lvert E_t\rvert)$, with TTL decay actively shrinking E_t	$n \cdot d$ floats + graph overhead of the index

Two honesty notes. First, a warm ANN index answers queries in empirically logarithmic time with excellent recall [26]; the query-time asymptotics are not where the categorical gap is. The gap is (i) the $\Theta(d)$ factor per comparison versus $\Theta(1)$, (ii) the encoder forward pass on the query path, which typically dominates end-to-end latency, and (iii) the existence of the refresh column at all. Second, Theorem 4.2's bound is distribution-free but per-query; a dense-retrieval system amortizes its training cost over queries, so for a *static* corpus at very high query volume the factorized reader can win on total cost. The comparison tilts to Lantern exactly in proportion to the write/query ratio and the drift rate.

6.2 Cold start

Proposition 6.1 (sample-size-free exactness). For any graph — including one with a single edge — the guarantees of Lemma 4.1 and Theorem 4.2 hold as stated; in particular a vertex is retrievable from the instant its first edge is written, with proximity equal to the asserted evidence (exactly, at one hop). No hypothesis on $\lvert E\rvert$, on the weight distribution, or on any generative model is used anywhere in §4.

Contrast the factorized reader at both of its levels:

- **Corpus level.** By §5.3, below $\Theta(nr \operatorname{polylog} n)$ observed relations the low-rank kernel is non-identifiable [29, 30]; a factorization fitted there returns noise with confident geometry.
- **Item level.** An interaction-trained embedding is undefined for an item with no interactions — the classical cold-start problem [42]. Content encoders sidestep this only for items that *have* content; entities defined purely by their relations (agents, sessions, keys in a shared working context — Lantern's MCP use case) offer nothing to encode.

Note the duality with §5.3: "exact from one edge" and "never invents similarity" are the same property read in two directions.

6.3 Non-stationarity

Lantern's kernel is a function of time by construction: Definition 2.1 makes every entry of A_t a sliding-window sum, each traversal evaluates at a single instant, and forgetting is not an operation but the default state of affairs ($O(1)$, per contribution, enforced by expiry). Formally, the direct reader computes top-k of $M(t)$ at the query's own t.

The factorized reader serves $\widehat{M} \approx M(t_0)$ and its error against the live kernel grows with the drift $\|M(t) - M(t_0)\|$ until the factorization cost is paid again. Three compounding frictions are documented in the literature:

1. **Refresh is global.** Incremental encoder updates or index inserts do not update the *kernel estimate*; only re-training does, at $\Omega(n)$.
2. **Embedding spaces are not stable across refreshes.** Retrained embeddings agree with their predecessors only up to (at best) an orthogonal transformation, which must be estimated (Procrustes alignment [37]); vectors from different training runs are not directly comparable, so a refresh forces a full re-encode + re-index rather than a rolling one.
3. **Deletion is adversarial to learned representations.** Removing one item's influence from trained parameters is the machine-unlearning problem [39, 40], and even at the index level, high-churn deletion in ANN structures requires periodic consolidation [38]. In Lantern, deletion *is* the resting state: evidence that is not re-asserted expires.

The scope of this advantage should be stated as carefully as its content: decay is the correct prior for *evidential, event-sourced* relations ("these two keys were co-accessed", "agent A cited fact B"), which age and should. It is the wrong prior for *stationary semantic* similarity ("cat" resembles "dog"), which does not decay and which an encoder captures zero-shot. The two readers are optimized for different stationarity regimes, and §7's recent systems increasingly combine them.

7. Related work

Proximity indices vs. latent factors. The comparison in this note is the systems-level descendant of the link-prediction literature: Liben-Nowell and Kleinberg [10] evaluated Katz, rooted PageRank, and common-neighbor indices directly against latent methods; Fouss et al. [9] used random-walk kernels (commute time, etc.) for collaborative recommendation, an early demonstration of graph-native proximity where factorization was the incumbent. Kernels on graphs as a general framework are due to Kondor and Lafferty [8].

Local graph algorithms. Personalized PageRank originates with the PageRank line [2, 3, 4]; local evaluation by forward-push and its $O(1/(\alpha\varepsilon))$ bound is Andersen–Chung–Lang [5] (independently Berkhin [6]); Monte-Carlo alternatives are analyzed in [48]. PageRank-Nibble [5] with the Lovász–Simonovits machinery [7] underlies `LocalCommunity`.

Embeddings as factorization. The identification of SGNS with shifted-PMI factorization is [12]; the closed forms for DeepWalk/LINE/node2vec are [22]; explicit kernel factorizations are HOPE [21] and

VERSE [23]; InstantEmbedding [24] closes the loop by *computing* embeddings from local PPR pushes. The rank-obstruction results are [32, 33].

ANN as graph search. NSW/HNSW [27, 26] and the theoretical treatment [28] establish that production vector search is greedy traversal of a similarity graph — the observation behind Proposition 3.4.

Graph-native retrieval for LLM memory. The regime of §6 — event-sourced, non-stationary, relation-defined entities — is precisely the agent-memory setting, and recent systems have converged on graph proximity there. HippoRAG [44] retrieves by running **Personalized PageRank** over an LLM-extracted knowledge graph, outperforming dense retrievers on multi-hop association at 10–20× lower cost; HippoRAG 2 [45] fuses the PPR reader with dense passage scores. GraphRAG [43] organizes corpus memory by community detection (cf. `LocalCommunity`); LightRAG [47] runs dual-level graph + vector retrieval. Zep's Graphiti [46] is the closest architectural relative of Lantern's temporal design: a bi-temporal knowledge-graph memory whose edges carry explicit validity intervals — the interval-validity counterpart of Definition 2.1's expiring contributions. Lantern's position in this landscape is the storage-engine one: a minimal $G = (V, E, w)$ substrate with decay and certified local readers, on which such memory systems can be built.

8. Limitations and scope

- **Generalization is real.** The factorized reader retrieves semantically similar items that share no path; the direct reader cannot, by design (§5.3). Where relations are abundant and stationary and serendipity is the product, embeddings are the right tool.
- **Directed-case guarantees.** Theorems 4.3 and the sweep-cut guarantee are proven for reversible chains; on Lantern's directed graphs they are inherited procedures with exact invariants (Lemma 4.1) but heuristic entrywise bounds (Remark 4.4). Sharpening this is open.
- **The kernel is only as good as the asserted edges.** Direct evaluation has no mechanism for repairing a mis-specified graph; the factorized reader's smoothing sometimes does exactly that.
- **Hybrids are the frontier, not a compromise.** An encoder can *propose* edges (a kNN graph over embeddings, Proposition 3.4) that Lantern then stores, decays, and traverses with certified local readers; HippoRAG 2 [45] and LightRAG [47] are evidence that the fused reader beats either alone. This composes naturally with Lantern's model because a proposed edge is just a contribution with a TTL.

9. Conclusion

"Lantern's neighborhood search" and "embedding + vector search" are not rival theories of similarity; they are rival *evaluation strategies* for one mathematical object, the walk kernel $M = \sum_k c_k P^k$. The embedding pipeline factorizes M once into $\mathbb{R}^{n\times d}$ and reads inner products; Lantern never factorizes and reads $M(t)$ itself, locally, at query time, with work independent of graph size. The correspondence is exact on near-low-rank symmetric kernels (and each reader literally implements the other's search procedure); the divergences are the rank/asymmetry obstructions, the completion-versus-fidelity trade, and — decisively for practice — the factorization stage's global cost, sample threshold, and frozen clock, none of which the direct reader pays. For event-sourced, decaying, relation-defined data, the direct reader dominates on all three operational axes; for stationary semantic content, the factorized reader's generalization is irreplaceable. The two compose, and the composition is where retrieval systems are visibly heading.

References

[1] L. Katz. *A new status index derived from sociometric analysis.* Psychometrika 18(1), 1953.

[2] L. Page, S. Brin, R. Motwani, T. Winograd. *The PageRank citation ranking: Bringing order to the Web.* Stanford InfoLab TR, 1999.

[3] T. Haveliwala. *Topic-sensitive PageRank.* WWW 2002.

[4] G. Jeh, J. Widom. *Scaling personalized web search.* WWW 2003.

[5] R. Andersen, F. Chung, K. Lang. *Local graph partitioning using PageRank vectors.* FOCS 2006.

[6] P. Berkhin. *Bookmark-coloring algorithm for personalized PageRank computing.* Internet Mathematics 3(1), 2006.

[7] L. Lovász, M. Simonovits. *The mixing rate of Markov chains, an isoperimetric inequality, and computing the volume.* FOCS 1990.

[8] R. I. Kondor, J. Lafferty. *Diffusion kernels on graphs and other discrete structures.* ICML 2002.

[9] F. Fouss, A. Pirotte, J.-M. Renders, M. Saerens. *Random-walk computation of similarities between nodes of a graph with application to collaborative recommendation.* IEEE TKDE 19(3), 2007.

[10] D. Liben-Nowell, J. Kleinberg. *The link-prediction problem for social networks.* JASIST 58(7), 2007.

[11] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, J. Dean. *Distributed representations of words and phrases and their compositionality.* NeurIPS 2013.

[12] O. Levy, Y. Goldberg. *Neural word embedding as implicit matrix factorization.* NeurIPS 2014.

[13] J. Pennington, R. Socher, C. Manning. *GloVe: Global vectors for word representation.* EMNLP 2014.

[14] S. Arora, Y. Li, Y. Liang, T. Ma, A. Risteski. *A latent variable model approach to PMI-based word embeddings.* TACL 4, 2016.

[15] K. W. Church, P. Hanks. *Word association norms, mutual information, and lexicography.* Computational Linguistics 16(1), 1990.

[16] K. Spärck Jones. *A statistical interpretation of term specificity and its application in retrieval.* Journal of Documentation 28(1), 1972.

[17] S. Robertson, H. Zaragoza. *The probabilistic relevance framework: BM25 and beyond.* Foundations and Trends in IR 3(4), 2009.

[18] B. Perozzi, R. Al-Rfou, S. Skiena. *DeepWalk: Online learning of social representations.* KDD 2014.

[19] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, Q. Mei. *LINE: Large-scale information network embedding.* WWW 2015.

[20] A. Grover, J. Leskovec. *node2vec: Scalable feature learning for networks.* KDD 2016.

[21] M. Ou, P. Cui, J. Pei, Z. Zhang, W. Zhu. *Asymmetric transitivity preserving graph embedding (HOPE).* KDD 2016.

[22] J. Qiu, Y. Dong, H. Ma, J. Li, K. Wang, J. Tang. *Network embedding as matrix factorization: Unifying DeepWalk, LINE, PTE, and node2vec.* WSDM 2018. arXiv:1710.02971

[23] A. Tsitsulin, D. Mottin, P. Karras, E. Müller. *VERSE: Versatile graph embeddings from similarity measures.* WWW 2018.

[24] Ş. Postăvaru, A. Tsitsulin, F. M. G. de Almeida, Y. Tian, S. Lattanzi, B. Perozzi. *InstantEmbedding: Efficient local node representations.* 2020. arXiv:2010.06992

[25] M. Belkin, P. Niyogi. *Laplacian eigenmaps for dimensionality reduction and data representation.* Neural Computation 15(6), 2003.

[26] Y. Malkov, D. Yashunin. *Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs.* IEEE TPAMI 42(4), 2020. arXiv:1603.09320

[27] Y. Malkov, A. Ponomarenko, A. Logvinov, V. Krylov. *Approximate nearest neighbor algorithm based on navigable small world graphs.* Information Systems 45, 2014.

[28] L. Prokhorenkova, A. Shekhovtsov. *Graph-based nearest neighbor search: From practice to theory.* ICML 2020.

[29] E. Candès, B. Recht. *Exact matrix completion via convex optimization.* Foundations of Computational Mathematics 9, 2009.

[30] E. Candès, T. Tao. *The power of convex relaxation: Near-optimal matrix completion.* IEEE Trans. Information Theory 56(5), 2010.

[31] B. Recht. *A simpler approach to matrix completion.* JMLR 12, 2011.

[32] C. Seshadhri, A. Sharma, A. Stolman, A. Goel. *The impossibility of low-rank representations for triangle-rich complex networks.* PNAS 117(11), 2020. doi:10.1073/pnas.1911030117

[33] S. Chanpuriya, C. Musco, K. Sotiropoulos, C. Tsourakakis. *Node embeddings and exact low-rank representations of complex networks.* NeurIPS 2020.

[34] A. Tversky. *Features of similarity.* Psychological Review 84(4), 1977.

[35] L. Vilnis, A. McCallum. *Word representations via Gaussian embedding.* ICLR 2015.

[36] M. Nickel, D. Kiela. *Poincaré embeddings for learning hierarchical representations.* NeurIPS 2017.

[37] W. L. Hamilton, J. Leskovec, D. Jurafsky. *Diachronic word embeddings reveal statistical laws of semantic change.* ACL 2016.

[38] A. Singh, S. J. Subramanya, R. Krishnaswamy, H. V. Simhadri. *FreshDiskANN: A fast and accurate graph-based ANN index for streaming similarity search.* 2021. arXiv:2105.09613

[39] Y. Cao, J. Yang. *Towards making systems forget with machine unlearning.* IEEE S&P 2015.

[40] L. Bourtoule et al. *Machine unlearning.* IEEE S&P 2021.

[41] E. Cohen, M. Strauss. *Maintaining time-decaying stream aggregates.* PODS 2003.

[42] A. Schein, A. Popescul, L. Ungar, D. Pennock. *Methods and metrics for cold-start recommendations.* SIGIR 2002.

[43] D. Edge et al. *From local to global: A Graph RAG approach to query-focused summarization.* 2024. arXiv:2404.16130

[44] B. Jiménez Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, Y. Su. *HippoRAG: Neurobiologically inspired long-term memory for large language models.* NeurIPS 2024. arXiv:2405.14831

[45] B. Jiménez Gutiérrez, Y. Shu, W. Qi, S. Zhou, Y. Su. *From RAG to memory: Non-parametric continual learning for large language models (HippoRAG 2).* ICML 2025. arXiv:2502.14802

[46] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, D. Chalef. *Zep: A temporal knowledge graph architecture for agent memory.* 2025. arXiv:2501.13956

[47] Z. Guo, L. Xia, Y. Yu, T. Ao, C. Huang. *LightRAG: Simple and fast retrieval-augmented generation.* 2024. arXiv:2410.05779

[48] D. Fogaras, B. Rácz, K. Csalogány, T. Sarlós. *Towards scaling fully personalized PageRank: Algorithms, lower bounds, and experiments.* Internet Mathematics 2(3), 2005.

[49] G. H. Golub, C. F. Van Loan. *Matrix Computations.* 4th ed., JHU Press, 2013. (Neumann series; SVD background.)

[50] C. Eckart, G. Young. *The approximation of one matrix by another of lower rank.* Psychometrika 1(3), 1936.